

Industrial Oriented Mini Project Report (CM653PC)

on

**AI-Smart Public Safety System with Crime Prediction and
Emergency Response System**

Submitted

*in partial fulfillment of the requirements for
the award of the degree of*

Bachelor of Technology

in

**Computer Science and Engineering
(Artificial Intelligence & Machine Learning)**

by

SHIVAGNA PABBA (23261A6638)

A. UJWALA (24265A6601)

Under the guidance of

Dr. B. Yadaiah

Assistant Professor

Dr. T.V. Rajini Kanth

Professor & Convener R&D

Director of IIIT



Department of Emerging Technologies

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY

Gandipet, Hyderabad – 500075 Telangana, India

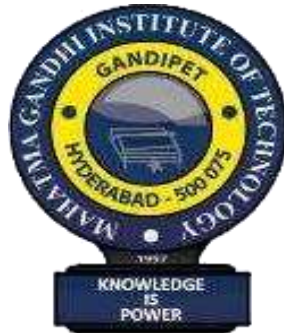
2025 – 2026

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY (Autonomous)

(Affiliated to Jawaharlal Nehru Technological University Hyderabad)

GANDIPET, HYDERABAD – 500075, Telangana

CERTIFICATE



This is to certify that the project entitled —**AI-Smart Public Safety System with Crime Prediction and Emergency Response System** is being submitted by **Ms. SHIVAGNA PABBA** bearing **Roll No: 23261A6638**, **Ms. A.UJWALA** bearing **Roll No. 24265A6601** in partial fulfilment for Industrial Oriented Mini Project in **COMPUTER SCIENCE AND ENGINEERING (AI and ML)** is a record of bonafide work carried out by them under the guidance and supervision.

The results of the investigations enclosed in this report have been verified and found satisfactory.

Project Guides

Dr. B.Yadaiah
Assistant Professor

Dr. T.V. Rajini Kanth
Professor & Convener R&D
Director of IIIT

Head Of The Department

Dr. M. Rama Bai
Professor & HOD
Department of ET

External Examiner

DECLARATION

This is to certify that the work reported in this project titled **AI-Smart Public Safety System with Crime Prediction and Emergency Response System** is a record of work done by us in the Department of Computer Science and Engineering (Artificial intelligence and Machine Learning), Mahatma Gandhi Institute of Technology, Hyderabad.

No part of the work is copied from books/journals/internet and wherever the portion is taken, the same has been duly referred in the text. The report is based on the work done entirely by us and not copied from any other source.

SHIVAGNA PABBA
23261A6638

A.UJWALA
24265A6601

ACKNOWLEDGEMENTS

We would like to express my sincere thanks to **Dr. G. ChandraMohan Reddy, Principal MGIT**, for providing the working facilities in college.

We wish to express my sincere thanks and gratitude to **Dr .M .Rama Bai, Professor and HoD**, Department of ET, MGIT, for all the timely support and valuable suggestions during the period of project.

We are extremely thankful to our Project Coordinators **Ms. J. Sreedevi, Assistant Professor**, and **Ms. S. Asha Jyothi, Assistant Professor** Department of ET, MGIT, industrial oriented mini project coordinators for their encouragement and support throughout the project.

We are extremely thankful and indebted to my internal guide **Dr. B. Yadaiah, Assistant Professor**, **Dr. T.V. Rajini Kanth, Professor & Converer R&D, Director of IIIL**, Department of ET, MGIT for their constant guidance, encouragement and moral support throughout the project.

Finally, we would also like to thank all the faculty and staff of ET Department who helped us directly or indirectly, for completing this project.

SHIVAGNA PABBA
23261A6638

A.UJWALA
24265A6601

TABLE OF CONTENTS

Certificate	i
Declaration	ii
Acknowledgement	iii
List of Figures	v
List of Tables	vi
Abstract	vii
1 Introduction	1
1.1 Motivation	1
1.2 Problem Definition	2
1.3 Existing System	2
1.4 Proposed System	3
1.5 Requirements Specification	3
2 Literature Survey	4
3 Design and Methodology	7
3.1 Design	7
3.2 Methodology	8
3.3 Tools and Technologies	10
4 Testing and Results	12
4.1 Testing	12
4.2 Results	13
5 Conclusion and Future work	21
5.1 Conclusion	21
5.2 Future work	22
BIBLIOGRAPHY	
APPENDIX	

LIST OF FIGURES

Figure No.	Figure Name	Page No
Figure 3.1	System Architecture of Crime Detection	7
Figure 3.2	Working Process of DBSCAN Cluster	11
Figure 4.1	Public User Registration Dashbord	13
Figure 4.2	Login into Crime Safety Dashboard	13
Figure 4.3	Crime Hotspot Map Visualization	14
Figure 4.4	Safe Route Finder	14
Figure 4.5	Aera Risk Checker	15
Figure 4.6	SOS Emergency Alert System	15
Figure 4.7	Nearby Polic Stations	16
Figure 4.8	Live Safety Alerts	16
Figure 4.9	Report an incident	17
Figure 4.10	Police Dashboard Overview	17
Figure 4.11	AI Patroa Recommendations	18
Figure 4.12	Tonight's Risk Prediction	18
Figure 4.13	SOS Alerts Monitoring	19
Figure 4.14	Marking SOS Alerts as Resolved	19
Figure 4.15	Incident Details with Live Location	20

LIST OF TABLES

Table No.	Table Name	Page No
Table 2.1	Literature survey on Crime Detection	6

ABSTRACT

In today's urban environment, ensuring public safety has become a major challenge due to the increasing number of crime incidents and lack of real-time awareness among citizens. People often do not know which areas are safe, especially during night time, and emergency response systems are not always quick or efficient. To address this issue, this project proposes an AI-Smart Public Safety System that helps users understand crime-prone areas, predict risk levels, and take safer decisions. The main objective of this system is to improve public safety by providing features like crime risk prediction, safe route suggestions, SOS emergency alerts, and support for police decision-making.

The system works by using a combination of machine learning and data analysis techniques. A dataset of crime records is used, and important features such as location, time, crime type, and severity are extracted. A Random Forest model is applied to predict the risk level of a particular area, while DBSCAN clustering is used to identify crime hotspots. The system processes user inputs like source and destination for safe route analysis or area and time for risk prediction. In case of emergencies, the SOS feature detects the user's location and identifies the nearest police station, sending alerts and storing the information in the database. Users can also report incidents, which are saved and used for further analysis. The backend handles all processing through APIs, while the frontend displays results in a user-friendly dashboard.

The system provides accurate risk predictions, identifies high-risk areas, and suggests safer routes, helping users avoid dangerous locations. It also enables quick emergency response by notifying nearby police stations and supports law enforcement through crime analysis and patrol recommendations. Overall, this project demonstrates how AI and machine learning can be effectively used to enhance public safety. It is a practical and scalable solution that can be further improved by integrating real-time data and advanced technologies in the future.

1. INTRODUCTION

Public safety and crime management are important areas in today's society, especially in urban environments where population density and daily activities are high. Ensuring the safety of citizens and providing timely support during emergencies is a critical responsibility for both government and technology systems. With the advancement of Artificial Intelligence and data analytics, there is a growing opportunity to use technology to improve safety, awareness, and decision-making [3][5]. This field matters because it directly impacts people's lives, helping them avoid dangerous situations and enabling authorities to respond more effectively.

However, one major issue is that people do not have access to real-time information about crime-prone areas, and existing systems mainly focus on reporting incidents after they happen rather than preventing them [9]. Current solutions lack predictive capabilities, do not provide safe route suggestions, and often have slow emergency response mechanisms. Additionally, there is limited integration between public users and police systems, making it difficult to take quick and informed actions. Because of these limitations, users are unable to make safe decisions while travelling, and law enforcement lacks proper tools for proactive crime prevention.

To overcome these challenges, this project proposes an AI-Smart Public Safety System with Crime Prediction and Emergency Response capabilities. The system aims to predict crime risk using machine learning, identify crime hotspots using clustering techniques, and provide features such as safe route finding, area risk analysis, SOS alerts, and incident reporting. It also supports police with data-driven insights and patrol recommendations. By combining AI techniques with a user-friendly interface and real-world data sources, the proposed system helps both citizens and authorities take smarter and faster decisions, ultimately improving overall public safety.

1.1 Motivation

In our daily life, safety has become a very important concern, especially in cities where crime cases are increasing. Many people travel alone for studies, jobs, or personal work, but they are not aware of which areas are safe and which are risky, particularly during night time. Also, in emergency situations, it is difficult to get quick help or contact nearby police stations. Existing systems mostly provide information after an incident happens and do help in preventing it

beforehand.

Because of this, there is a need for a smart system that can guide people in real-time and help them make safer decisions. By using Artificial Intelligence and data analysis, it is possible to predict crime risk, identify unsafe areas, and provide useful features like safe route suggestions and SOS alerts. This project focuses on building such a system that can support both public users and police, making the environment safer and more secure.

1.2 Problem Definition

Crime incidents in urban areas are increasing, and people often do not have proper information about which places are safe or unsafe. Existing systems mainly focus on reporting crimes after they happen and do not provide real-time safety guidance or prediction. This leads to unsafe travel decisions and delayed emergency responses, especially during night time. There is a need for a smart system that can predict risk and help users stay safe. The proposed AI-Smart Public Safety System with Crime Prediction and Emergency Response System provides solutions like risk prediction, safe routes, and SOS support.

1.3 Existing System

The current systems for crime analysis mainly rely on historical data presented in the form of reports, charts, or basic maps. These systems are mostly used by police departments and are not easily accessible to the general public. They focus more on analyzing past incidents rather than providing real-time guidance or predictive insights. Because of this, users cannot make informed decisions about safety while travelling or staying in a particular area. Although some modern methods use GIS-based mapping and basic data analytics to identify crime-prone regions, they still lack advanced machine learning techniques for accurate prediction. These systems do not consider important factors like time, location variations, or dynamic risk changes. Additionally, there is no proper integration of features like safe route suggestions, emergency SOS alerts, or user interaction through dashboards. Due to these limitations, there is a clear gap in providing a complete, real-time, and user-friendly safety solution. Existing systems are not proactive and fail to support both citizens and police effectively.

1.4 Proposed System

The proposed system aims to improve public safety by providing real-time crime risk analysis and emergency support. It uses machine learning techniques like Random Forest for risk prediction and DBSCAN for hotspot detection. The system works by analyzing user inputs such as location, time, and route, and processes crime data to generate risk levels and safety insights. Key features include safe route finder, area risk analysis, SOS alerts, and incident reporting. It offers advantages like better decision-making, faster emergency response, and improved awareness. The system is accessible through a user-friendly web application and helps both citizens and police, creating a safer and smarter environment.

1.5 Requirements Specification

1.5.1 Software Requirements

Language	: Python, JavaScript(React)
Operating system	: Windows 10
IDE	: Microsoft Visual Code
Libraries/Tools	: Flask, Pandas, NumPy, Scikit-learn, DBSCAN, Pickle, Axios, React, Leaflet (Maps)
Database	: SQLite
Browser	: Any modern web browser (Chrome, Firefox, Edge)

Note: Active Internet Connection is required for map display and API communication.

1.5.2 Hardware Requirements:

RAM	: 8GB minimum
ROM	: 256GB minimum
Processor	: Intel Core-i5 minimum
Optional	: GPU (for future ML enhancements), GPS-enabled device for location access
Display	: Monitor or mobile screen to use the system

2. LITERATURE SURVEY

A literature survey on crime analysis and public safety systems highlights the significant role of artificial intelligence and machine learning in improving crime prediction and urban safety management. Traditional methods mainly relied on manual crime reporting and basic statistical analysis, which often resulted in delayed decision-making and lack of real-time awareness among citizens. Recent advancements in machine learning and data analytics have enabled automated crime prediction, hotspot detection, and better understanding of crime patterns using historical data. Many studies have explored techniques like Random Forest, clustering algorithms such as DBSCAN, and data visualization tools for crime analysis. These approaches have shown promising results in identifying high-risk areas and supporting police decision-making. However, challenges such as lack of real-time user interaction, absence of safe route guidance, limited integration of emergency response systems, and focus on only a single functionality (like prediction or visualization) still exist. Moreover, many systems are either designed only for police or only for analysis purposes, without providing a complete solution for both public users and law enforcement.

[1] Wang et al. used machine learning algorithms to predict crime based on historical crime data, showing that data-driven models can identify crime patterns effectively. However, their system focused only on prediction and lacked real-time user interaction, safe route guidance, and emergency support features.

[2] Chainey and Ratcliffe applied clustering techniques like DBSCAN to identify crime hotspots, helping in understanding high-risk areas based on location data. However, their work was limited to hotspot detection and did not include crime risk prediction, user-level applications, or integration with police systems.

[3] Sharma et al. developed a smart policing system that helps law enforcement agencies analyse crime data and make decisions. However, the system was mainly designed for police use and did not provide any direct interaction for public users, nor did it include real-time alerts or safe navigation features.

[4] **Kumar et al.** designed an emergency response system using GPS technology to send user location details to authorities during emergencies. However, their system only focused on location tracking and did not include crime analysis, risk prediction, or intelligent nearest police station identification.

[5] **Zhang et al.** proposed a route recommendation system that suggests paths based on shortest distance and traffic conditions. However, the system ignored crime data and safety factors, making it unsuitable for users who need safer travel routes.

[6] **Andrienko et al.** developed crime data visualization systems using maps and charts to represent crime patterns. However, their work focused only on visualization and lacked predictive analytics, real-time alerts, and decision support mechanisms for users and police.

Table 2.1 Literature Survey on Crime Detection

S.No	Author Name	Title	Focus On	Limitations
1	Wang et al., 2018	Crime Prediction Using Machine Learning	ML models used to predict crime based on historical data	Only hotspot detection; no risk prediction or user interaction
2	Chainey & Ratcliffe, 2005	Crime Hotspot Detection Using Clustering	DBSCAN clustering to identify high crime areas	Only hotspot detection; no risk prediction or user interaction
3	Sharma et al., 2019	Smart Policing System	Crime data analysis tools for police decision-making	Focused only on police; no public access, no real-time alerts
4	Kumar et al., 2017	GPS-Based Emergency Response System	Sends user location to authorities during emergencies	Only location tracking; no crime prediction or nearest police logic
5	Zhang et al., 2016	Route Recommendation System	Suggests shortest path based on distance and traffic	Ignores crime data; no safety or risk-based routing
6	Andrienko et al., 2010	Crime Data Visualization System	Visualization of crime data using maps and charts	Only visualization; no prediction, no alerts, no decision support

3. DESIGN AND METHODOLOGY

3.1 Design

The system starts with collecting and preprocessing crime data using features like time, location, and severity. Machine learning models such as DBSCAN identify crime hotspots, while Random Forest predicts risk levels. The backend processes this information through APIs and stores it in an SQLite database. The frontend provides dashboards for public users and police to access the data.

It finally delivers outputs like safe routes, risk scores, live alerts, and emergency support.

System Architecture:

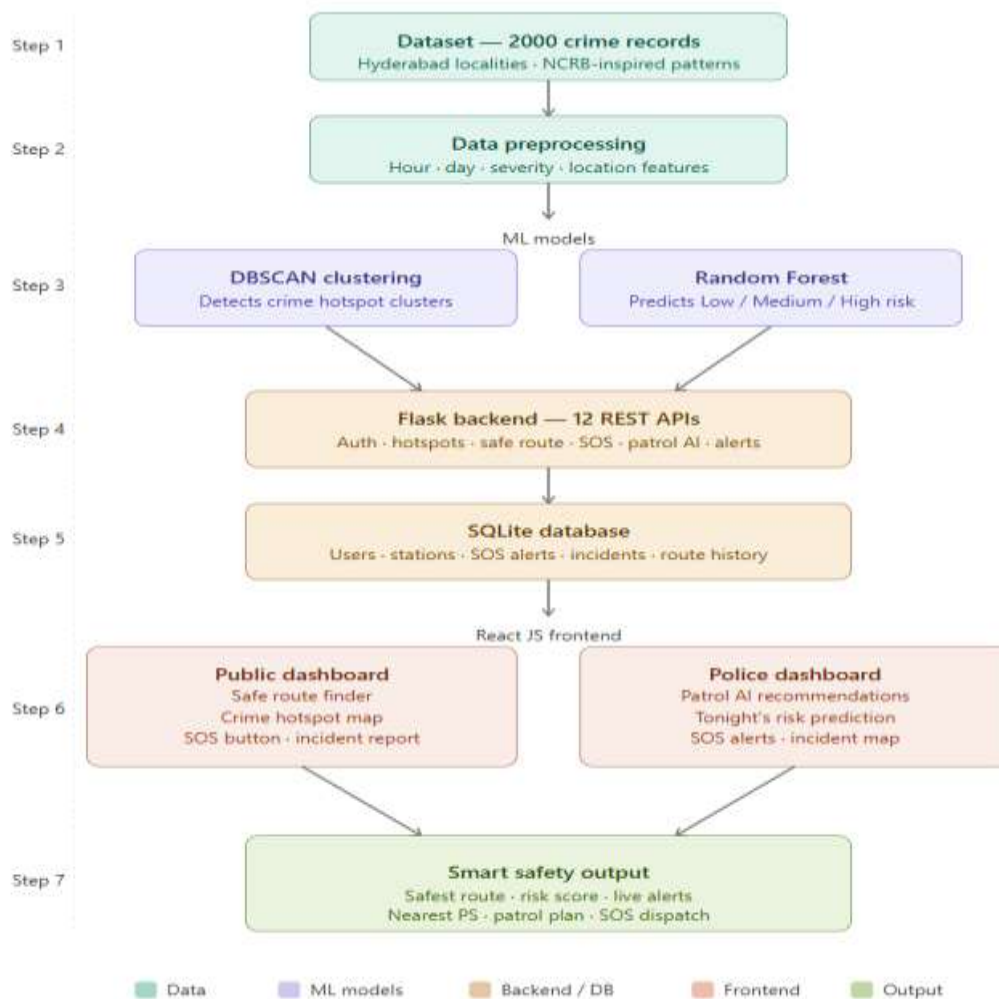


Figure 3.1 System Architecture of Crime Detection

3.2 Methodology

The AI-Smart Public Safety System was designed and developed as an intelligent, user-friendly web platform aimed at improving public safety through crime prediction and emergency response support. The system follows a structured and modular approach by integrating web technologies, machine learning algorithms, and backend APIs. This section explains the complete workflow of the system, starting from planning and design to deployment, along with the key modules used in development.

Planning and Design:

The initial phase focused on understanding user requirements such as safety awareness, easy navigation, quick risk prediction, and emergency support. Based on these requirements, a simple and responsive front-end interface was designed. The system includes modules like login/register, dashboard, crime map, safe route finder, area risk analysis, SOS alert, and incident reporting.

The interface design ensures that users can easily access all features. Separate dashboards were designed for public users and police users. The layout was planned in such a way that users can quickly check risk levels and take necessary actions. The overall design focuses on clarity, usability, and real-time interaction.

Development:

The backend of the system was developed using Python and the Flask web framework. Flask was used to create REST APIs for handling different functionalities such as login, risk prediction, safe route calculation, SOS alerts, and incident reporting.

The system follows a modular structure where each API handles a specific task. User authentication is implemented securely, and different roles (public and police) are managed. The backend processes user requests, interacts with the machine learning model, and returns results to the frontend. The frontend (React-based UI) communicates with the backend using API calls, making the system dynamic and interactive.

Data Processing and Prediction:

The system uses a crime dataset that contains details such as location, time, crime type, and severity. Before applying machine learning, the data is pre-processed by extracting useful features like hour, day, and night-time indicators.

A Random Forest model is used to predict the risk level (High, Medium, Low) of a particular area. The system also calculates additional parameters such as crime density and time-based risk. For safe route analysis, the system checks nearby crimes within a certain distance and calculates a total risk score. The final result is displayed to the user in real-time

Hotspot Detection:

To identify high-risk areas, the system uses the DBSCAN clustering algorithm. This algorithm groups nearby crime locations and identifies dense regions as crime hotspots. These hotspots are pre-processed and stored, and later used during analysis for improving prediction accuracy and visualization. This helps both users and police to understand which areas are more dangerous and require attention.

Emergency (SOS) System:

The system includes an SOS feature for emergency situations. When a user triggers SOS, the system collects the user's location and identifies the nearest police station using distance calculation. The alert is then saved in the database and sent to the police dashboard. This ensures quick response and proper tracking of emergency cases. The system also allows police users to view and manage SOS alerts.

Incident Reporting:

Users can report incidents by providing details such as crime type, location, and description. The system assigns the nearest police station and saves the report in the database. An alert is generated for the police, allowing them to take necessary action. This feature helps in maintaining updated crime data and improves system accuracy over time.

Database Management:

A SQLite database is used to store important data such as user details, SOS alerts, and reported incidents. The database ensures proper data organization and retrieval. Only critical information like user records, emergency alerts, and incident reports are stored. Machine learning predictions and safe route results are generated in real-time and are not stored permanently.

Testing and Deployment:

After development, the system was tested to ensure all features work correctly. This includes testing login functionality, risk prediction, safe route calculation, SOS alerts, and incident reporting. The system was deployed locally for demonstration purposes. It can be further extended for cloud deployment and real-time integration with external services.

3.3 Tools and Technologies

Flask (Python)

Flask is a lightweight Python web framework used to build the backend of the system. It handles API creation, request processing, and server-side logic. Flask's simplicity and flexibility make it suitable for rapid development and integration with machine learning models.

SQLite:

SQLite is a lightweight, serverless database used for storing system data. It stores user details, SOS alerts, and incident reports. It is easy to set up and works efficiently for local applications.

Machine Learning (Random Forest)

Random Forest is used for predicting crime risk levels based on input features like location, time, and crime type. It provides good accuracy and handles multiple features effectively, making it suitable for this application.

DBSCAN(Clustering Algorithm)

DBSCAN is used for crime hotspot detection. It groups nearby crime locations and identifies high-density areas. This helps in understanding crime patterns and improving prediction.

React(Frontend)

React is used to build the frontend interface of the system. It provides a dynamic and interactive user experience. It communicates with the backend through APIs and displays results such as maps, alerts, and predictions.

HTML and CSS

HTML and CSS are used for structuring and styling the frontend components. They ensure a clean and user-friendly interface for easy navigation.

Python Libraries (Pandas, Numpy, Scikit-learn)

These libraries are used for data preprocessing, feature extraction, and machine learning model implementation. They help in efficient data handling and model training.

DBSCAN Clustering for Crime Hotspot Detection

The system uses **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)** to identify crime hotspots. DBSCAN groups nearby crime locations based on density and marks regions with a high number of incidents as clusters. Points that do not belong to any cluster are treated as noise.

This method is highly effective for geographical data because it does not require specifying the number of clusters in advance. It works using two important parameters: epsilon (distance threshold) and minimum points required to form a cluster.

In this project, DBSCAN is used to detect areas where crimes occur frequently. These hotspots are pre-processed and stored, and later used for visualization and improving prediction accuracy. This helps both users and police identify high-risk zones and take preventive actions.

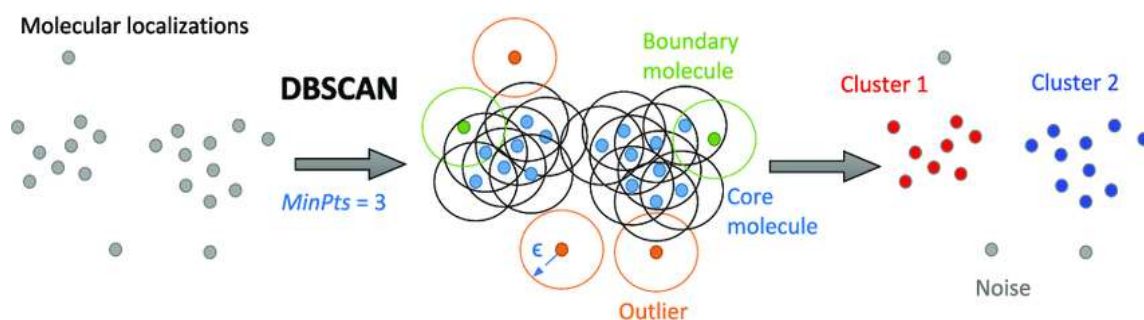


Figure 3.2 Working Process of DBSCAN Cluster

4. TESTING AND RESULTS

4.1 Testing

Testing is an important phase in the development of the AI-Smart Public Safety System to ensure accuracy, reliability, and proper functionality of all modules. The system was tested to verify that the machine learning model correctly predicts crime risk levels and that all features like safe route analysis, SOS alerts, and incident reporting work as expected.

The testing process included both frontend and backend validation, API testing, and real-time user interaction scenarios. Different test cases were considered such as valid and invalid inputs, incorrect login details, and edge cases like empty inputs or unavailable locations.

The system was tested using sample crime datasets and different input scenarios such as various locations, times, and crime types. Functional testing was performed for modules like login, safe route finder, area risk prediction, SOS alerts, and incident reporting.

Performance testing was carried out to evaluate response time and prediction speed. The system provides fast results due to efficient model integration. Usability testing ensured that the interface is simple, clear, and easy to use for general users.

Unit testing

Unit testing was performed to validate individual components of the system. Each module such as risk prediction, safe route calculation, SOS alert generation, and database operations was tested separately.

The machine learning model was tested with multiple inputs to ensure consistent prediction accuracy. API endpoints were tested individually to verify correct request handling and response generation.

Database operations such as storing users, SOS alerts, and incidents were also tested to ensure proper data storage and retrieval. This ensured that each component works correctly before integrating into the full system.

4.2 Results

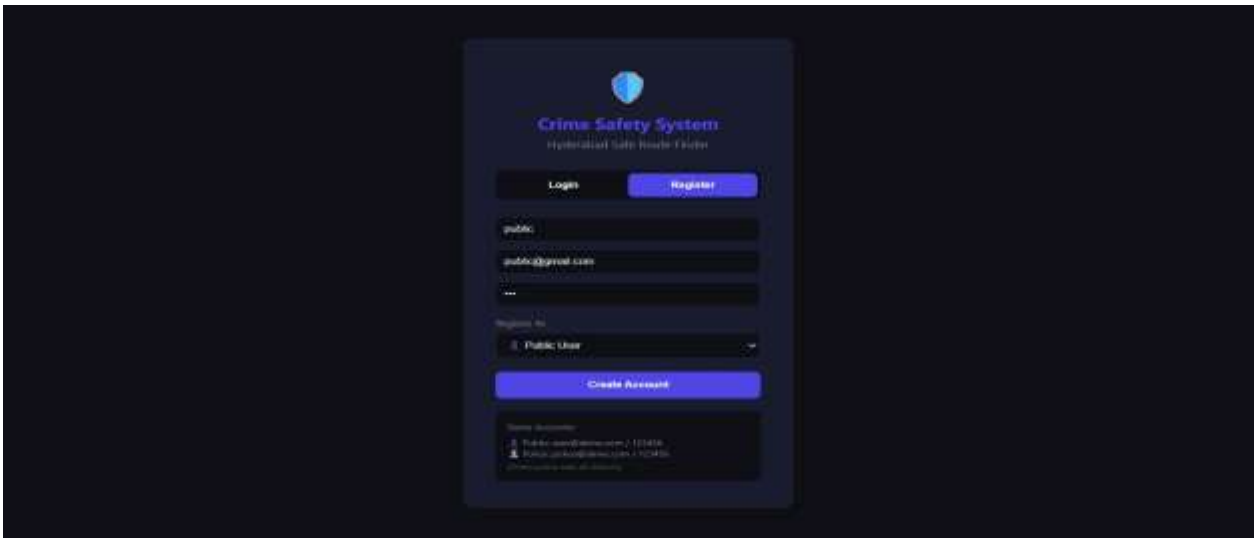


Figure 4.1: Public User Registration Dashboard

The above figure 4.1 shows the public user registration interface of the Crime Safety System. It allows new users to create an account by entering their name, email, password, and selecting the role as a public user. The interface also provides options to switch between login and registration. This dashboard ensures secure and easy access for users to utilize features like safe route finding and crime alerts.

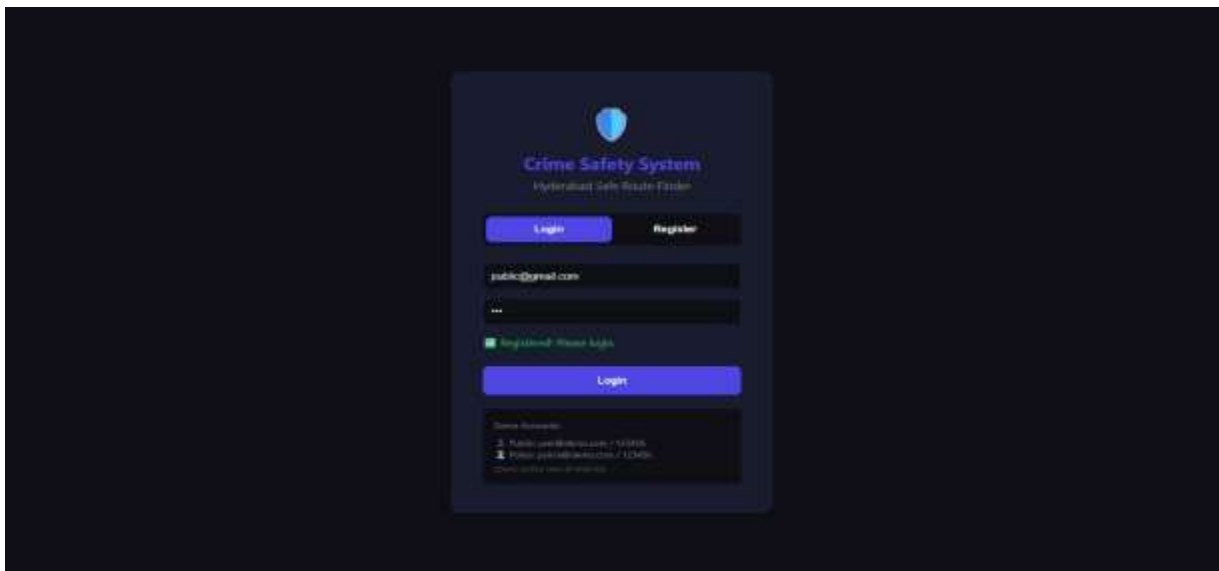


Figure 4.2: Login into Crime Safety System

The above figure 4.2 shows the login interface of the Crime Safety System. It allows registered users to enter their email and password to access the system. A confirmation message indicates successful registration and prompts the user to log in. This interface ensures secure authentication and provides access to features like crime analysis, safe route suggestions, and alerts.

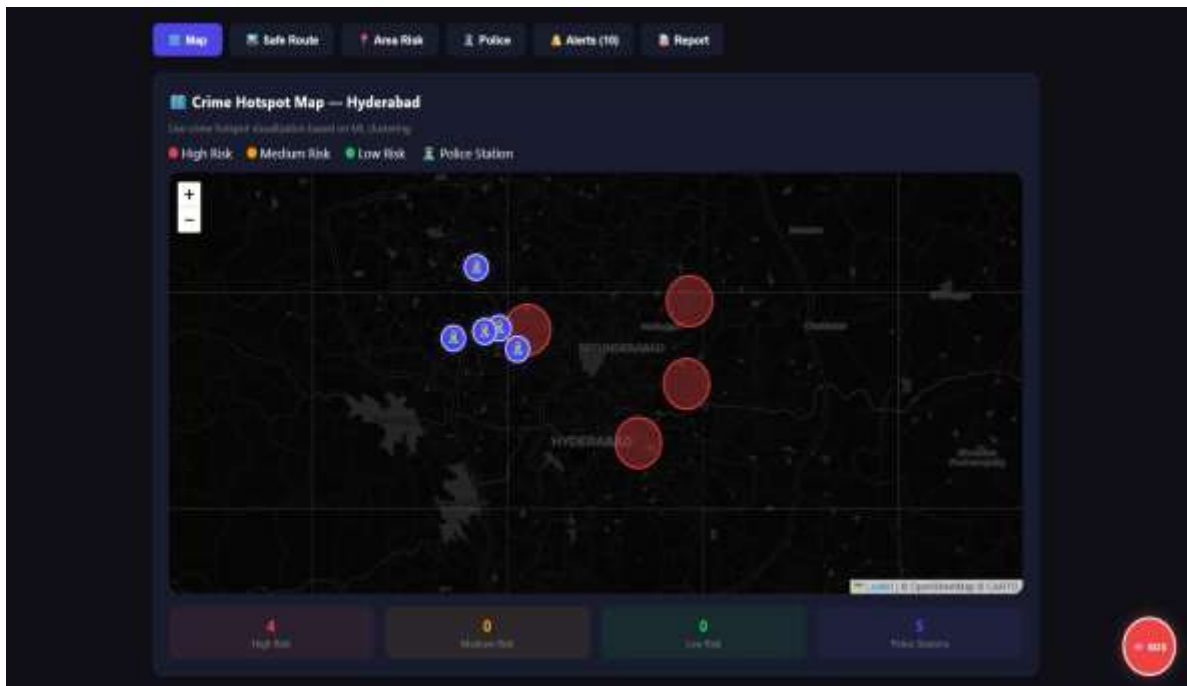


Figure 4.3: Crime Hotspot Map Visualization

The above figure 4.3 shows the crime hotspot map of Hyderabad, which visualizes different risk levels based on crime data analysis. The system highlights high-risk areas in red, helping users easily identify dangerous zones. It also displays police stations and other risk levels such as medium and low for better awareness. This map helps users and police to understand crime distribution and take safer decisions.

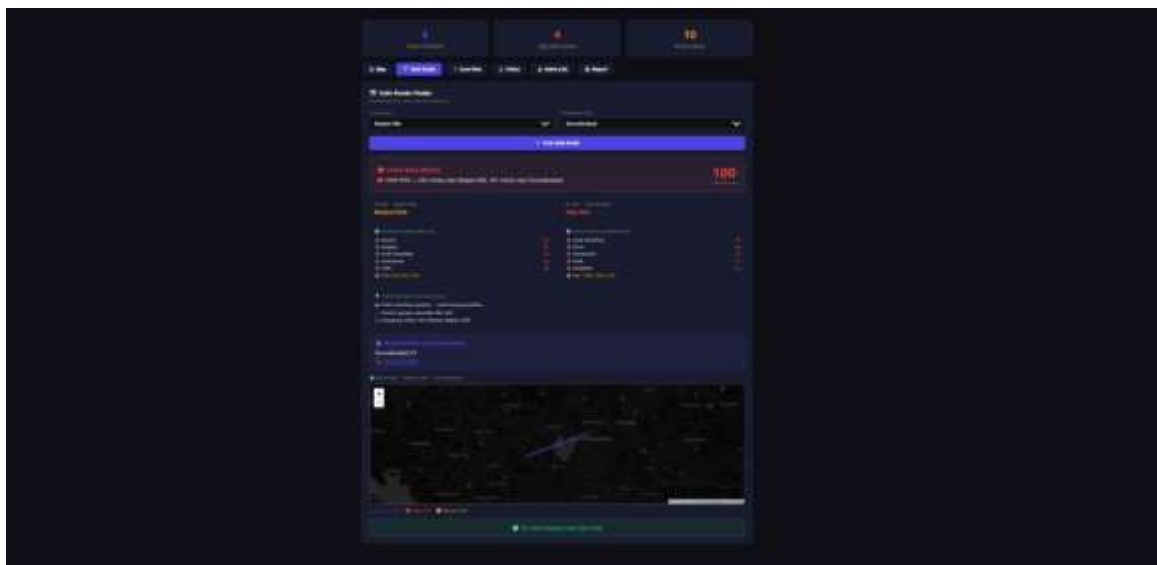


Figure 4.4: Safe Route Finder

The above figure 4.4 shows the safe route finder feature of the Crime Safety System, which helps users find a secure path between two locations. In this example, the route is calculated from Banjara Hills to

Secunderabad, displaying the risk levels of both areas. The system highlights a high-risk route and provides detailed crime information along with safety tips. It also shows a route map that prioritizes safety over time, helping users choose a safer path instead of the fastest one.

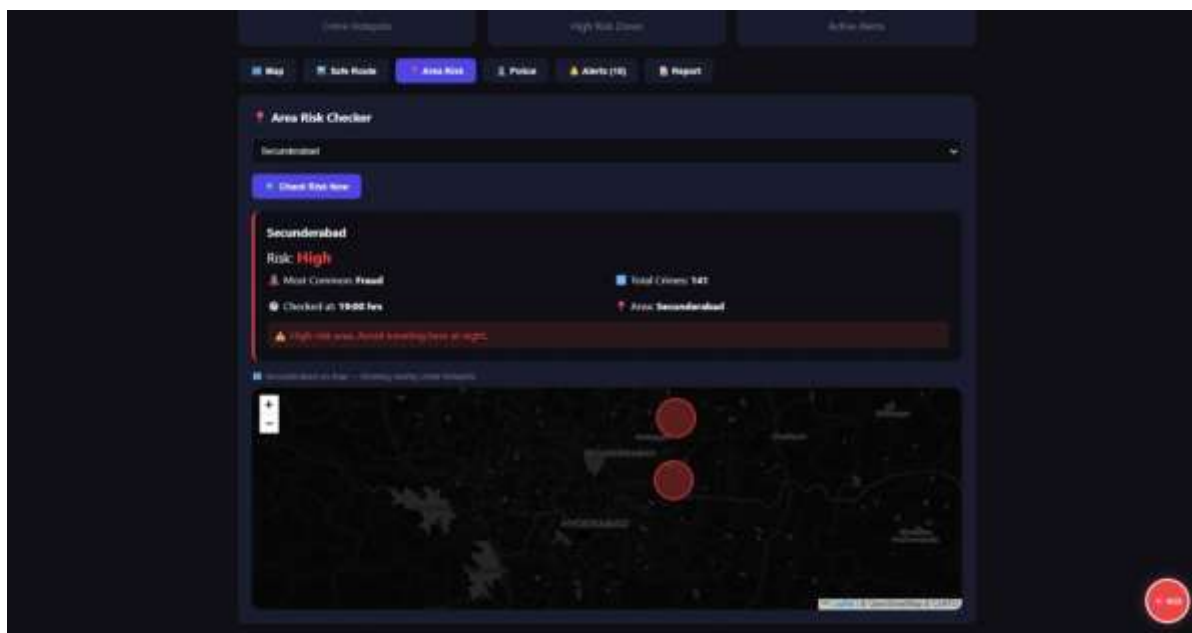


Figure 4.5: Area Risk Checker

The above figure 4.5 shows the area risk checking feature of the Crime Safety System. It allows users to select a location, such as Secunderabad, and view its risk level based on crime data. The system displays important details like total crimes, most common crime type, and the time of analysis. It also highlights the area on the map with nearby hotspots, helping users understand how safe or risky the location is.

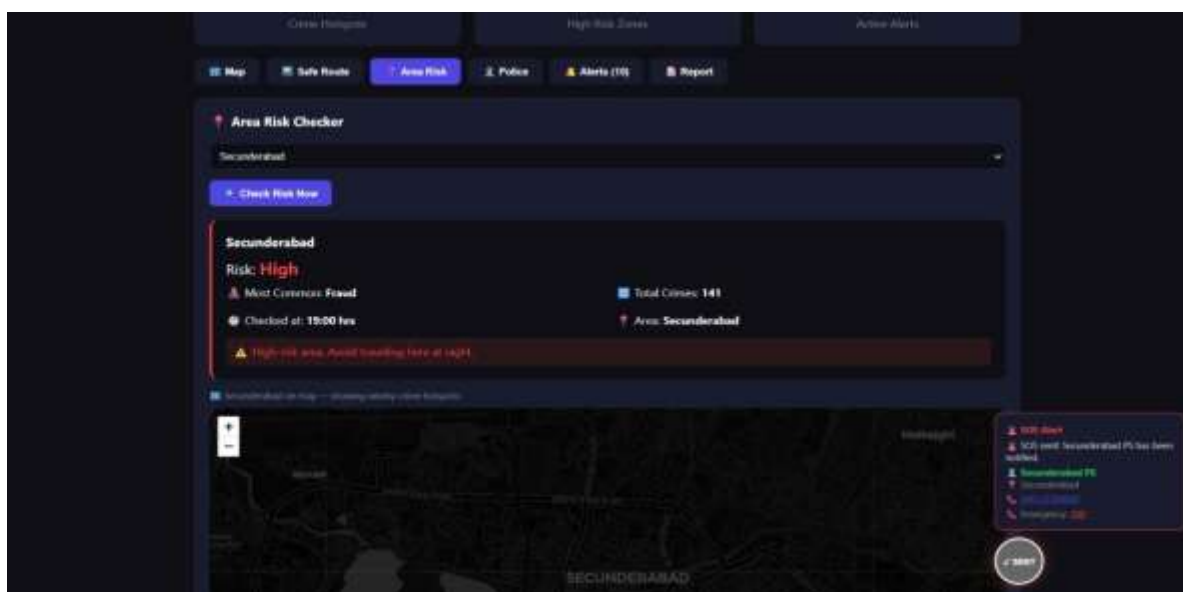


Figure 4.6: SOS Emergency Alert System

The above figure 4.6 shows the SOS emergency feature of the Crime Safety System. When the user presses the red SOS button during an emergency, an alert is immediately sent to the nearest police station. In this example, the user's location is Secunderabad, and the alert has been successfully sent to Secunderabad Police Station. This feature ensures quick response and immediate assistance in critical situations.

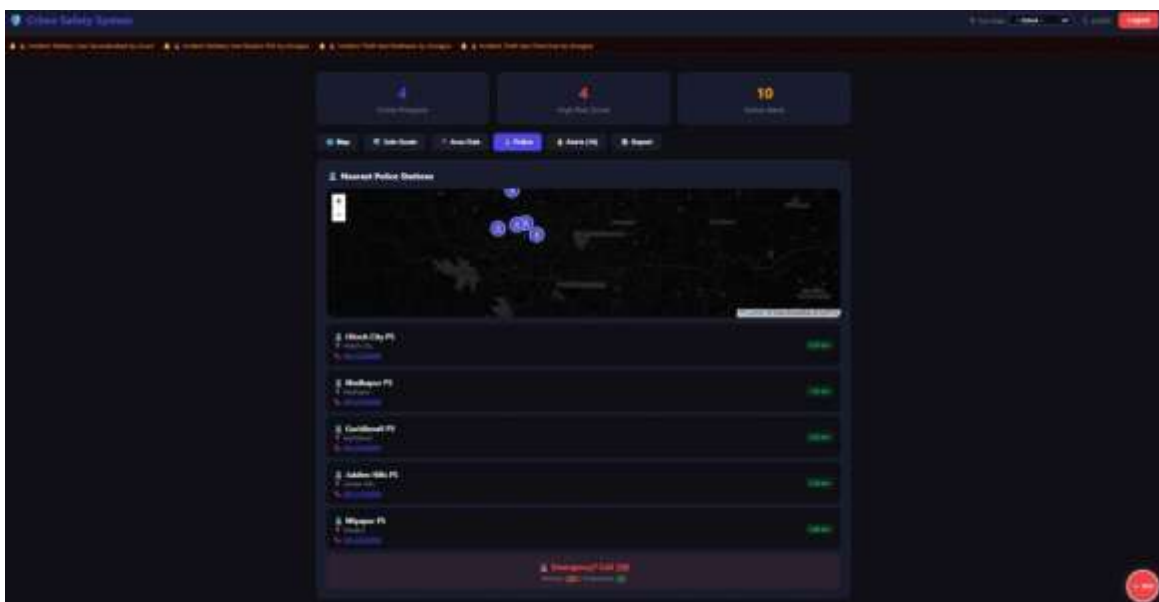


Figure 4.7: Nearby Police Stations

The above figure 4.7 shows the nearby police stations feature of the Crime Safety System. It displays all the closest police stations on the map along with their locations. The system also provides details such as station names, contact numbers, and distance from the user's location. This helps users quickly find and reach the nearest police station in case of need.

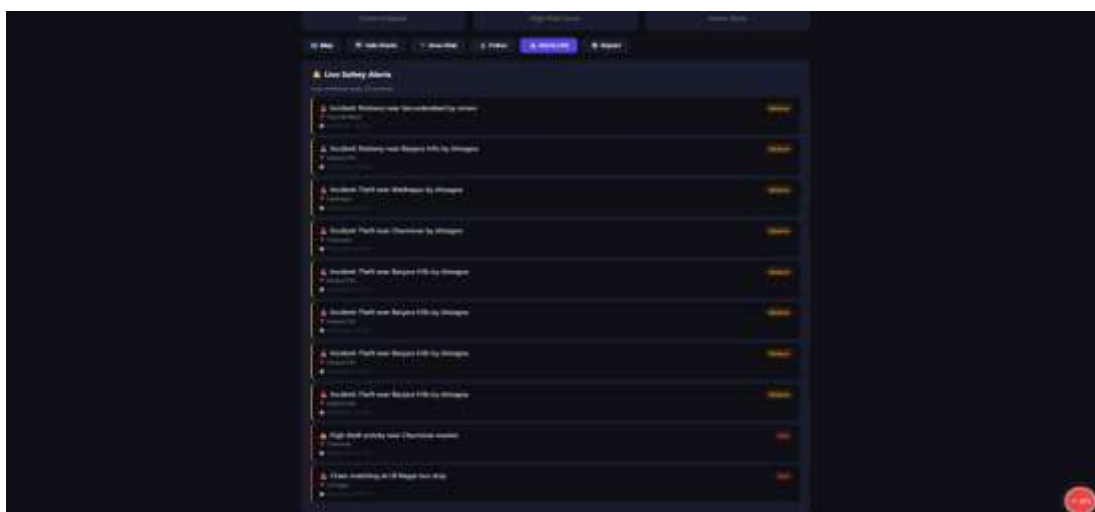


Figure 4.8: Live Safety Alerts

The above figure 4.8 shows the live safety alerts feature of the Crime Safety System. It displays recent incidents and alerts generated by users, including those triggered through the SOS emergency button. Each alert shows details like location, type of incident, time, and risk level. This helps users stay aware of nearby dangers and take precautions for their safety.

The screenshot shows the 'Report an Incident' form in the Crime Safety System. At the top, there are three summary cards: 'Crime Incidents' with a count of 4, 'High Risk Zones' with a count of 4, and 'Active Alerts' with a count of 10. Below these are navigation tabs: 'Map', 'Safe Route', 'Area Risk', 'Police', 'Alerts (18)', and 'Report'. The 'Report' tab is active, showing a form with the following fields: 'Crime Type' (dropdown menu with 'Vehicle Theft' selected), 'Nearby Location' (dropdown menu with 'Commercial' selected), 'Severity' (dropdown menu with 'Urgent' selected), and 'Description' (text area with the placeholder text 'I lost my vehicle, vehicle number is 2000 registered in Hyderabad'). A 'Submit Report' button is at the bottom of the form.

Figure 4.9: Report an Incident

The above figure 4.9 shows the incident reporting feature of the Crime Safety System. It allows users to report crimes by selecting the type, area, severity, and providing a description of the incident. Once submitted, the report is sent to the nearest police station for action. This feature also helps create awareness among other users about unsafe areas and improves overall community safety.

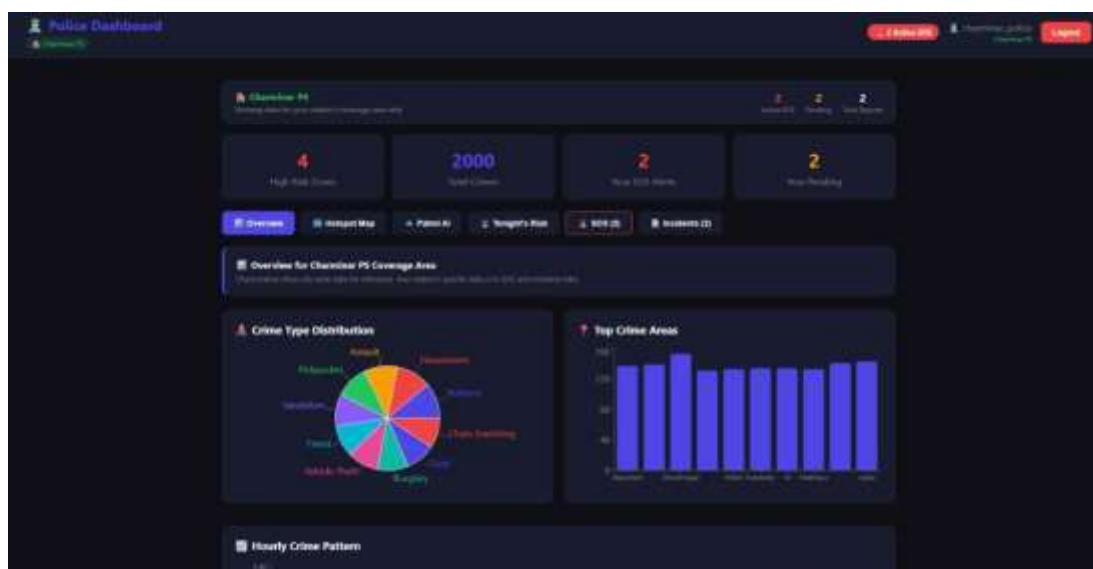


Figure 4.10: Police Dashboard Overview

The above figure 4.10 shows the police dashboard of the Crime Safety System. It provides an overview of crimes in the respective station area, including high-risk zones, total crimes, and active SOS alerts. The dashboard also visualizes crime type distribution and top crime areas using charts for easy understanding. This helps police officers quickly analyze crime patterns and make better decisions for safety and patrol planning.

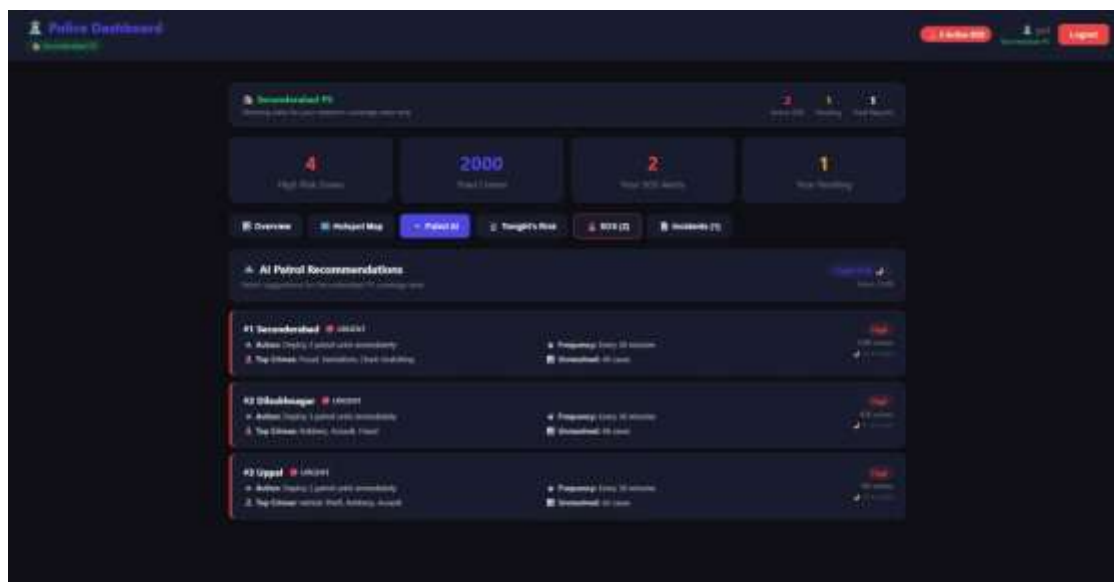


Figure 4.11: AI Patrol Recommendations

The above figure 4.11 shows the AI-based patrol recommendations in the police dashboard. It highlights specific areas where patrolling is required based on crime risk levels and recent data. The system suggests actions, patrol frequency, and priority levels for each area. This helps police officers efficiently plan patrols and focus on high-risk zones to improve safety.

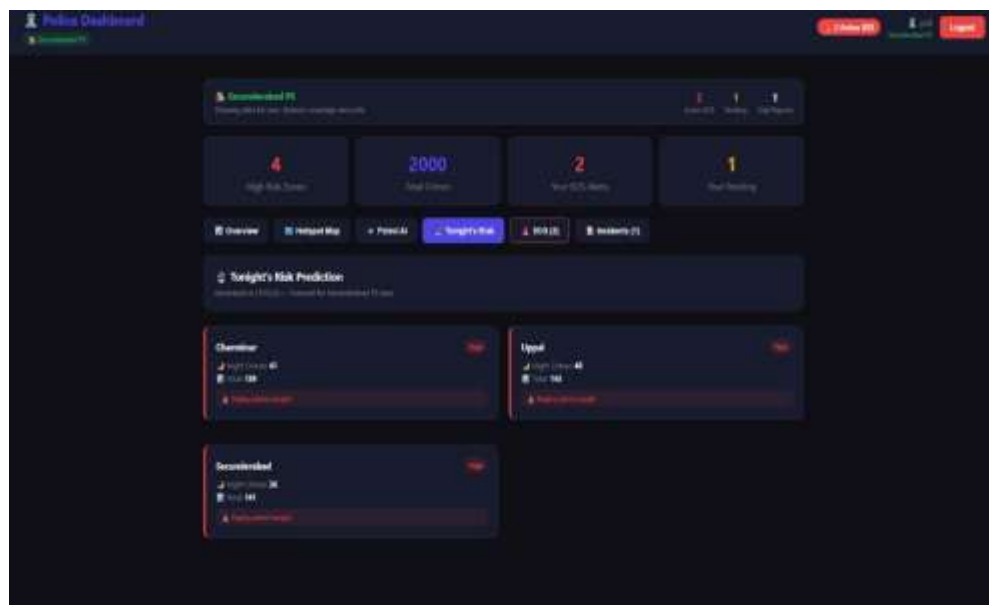


Figure 4.12: Tonight's Risk Prediction

The above figure 4.12 shows the night-time risk prediction feature in the police dashboard. It highlights nearby areas with high crime risk based on recent data analysis. The system displays details like number of night crimes and total crimes for each area. It also suggests deploying patrol units in these high-risk locations to improve safety during night hours.

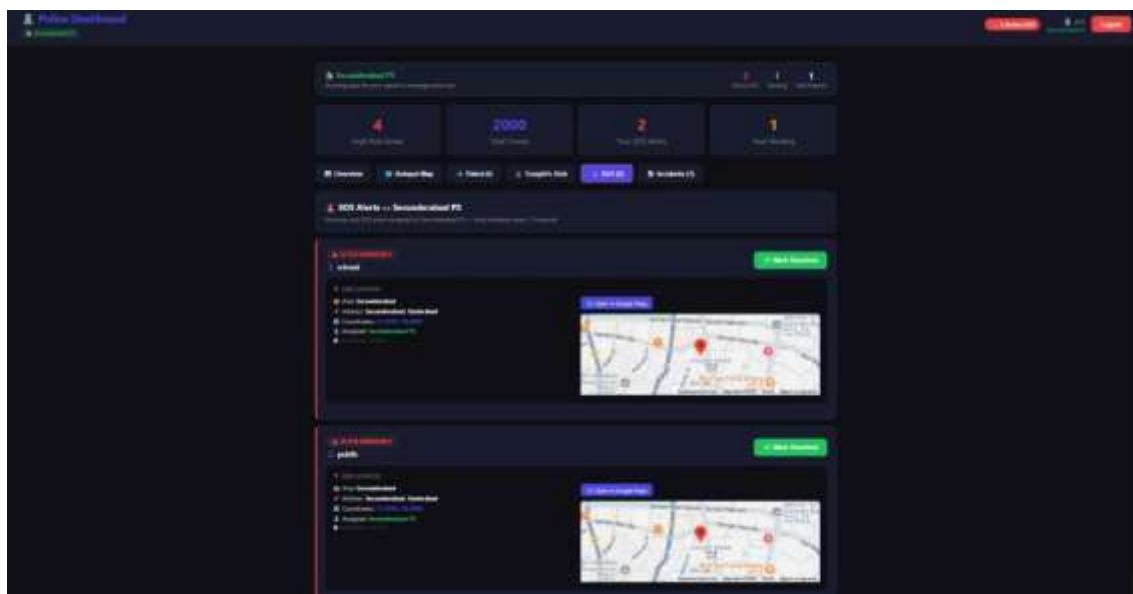


Figure 4.13: SOS Alerts Monitoring (Police Dashboard)

The above figure shows the SOS alerts section in the police dashboard. It displays the list of users who have sent emergency alerts along with their live location, area, date, and time. Police officers can view the exact location on the map and take immediate action. This feature helps in quick response and efficient handling of emergency.

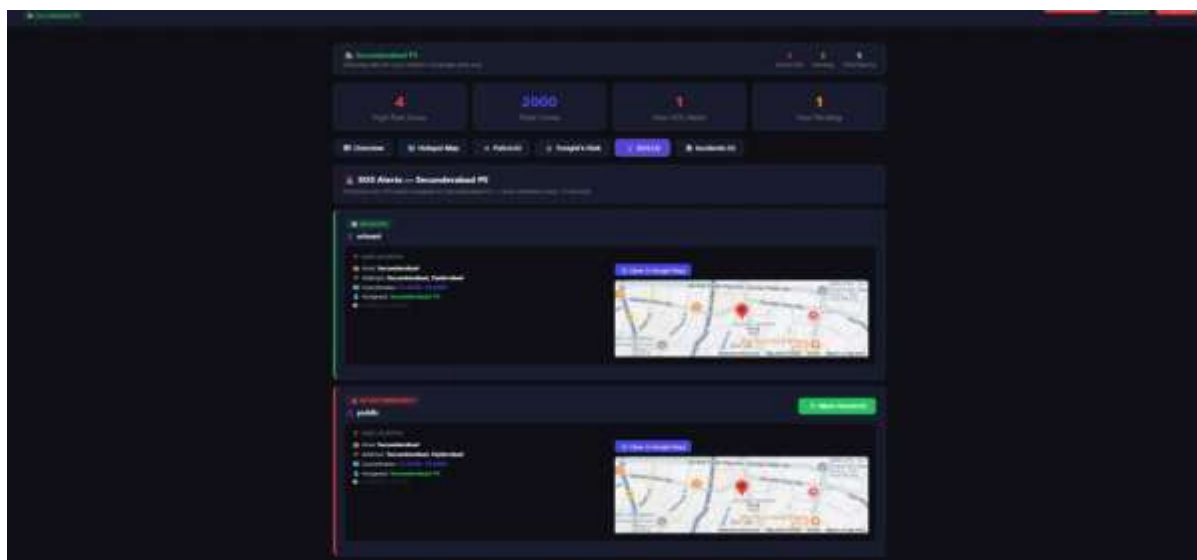


Figure 4.14: Marking SOS Alert as Resolved

The above figure 4.14 shows the feature to mark an SOS alert as resolved in the police dashboard. Once the emergency situation is handled, the police officer can click the “Mark Resolved” button. The alert status is then updated from active to resolved. This helps in tracking completed cases and managing ongoing emergencies efficiently.

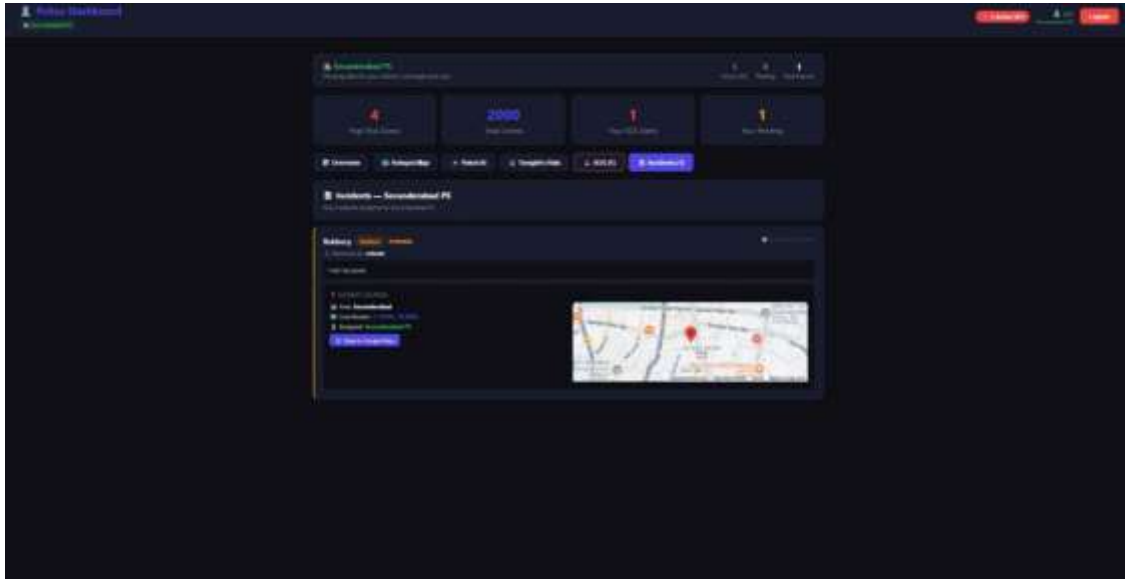


Figure 4.15: Incident Details with Live Location

The above figure shows the incident details section in the police dashboard after a vehicle theft is reported. It provides complete information about the incident, including type, description, area, and status. The system also shares the live location on the map to help police track the exact place of occurrence. This enables faster response and helps in identifying and catching the culprit effectively.

5. CONCLUSION AND FUTURE WORK

5.1 Conclusion

Our project, AI-Smart Public Safety System, is a highly useful solution designed to improve public safety by helping users identify crime-prone areas and take safer decisions. The main objective of this system is to provide an intelligent platform that predicts crime risk, suggests safe routes, and offers emergency support through SOS alerts. The application is simple and user-friendly, making it suitable for people of all backgrounds, including those with minimal technical knowledge.

The system integrates powerful machine learning techniques such as Random Forest for accurate risk prediction and DBSCAN for crime hotspot detection. These technologies work together with a responsive web interface to provide real-time insights about safety. Features like area risk analysis, safe route finder, incident reporting, and SOS alerts help users avoid dangerous situations and respond quickly during emergencies. This not only benefits the public but also supports police authorities in monitoring and managing crime effectively.

In addition to functionality, the project ensures proper data handling and system reliability. Important data such as user details, SOS alerts, and incident reports are securely stored, while predictions are generated in real time for efficiency. The system was thoroughly tested to ensure accuracy and smooth performance across all modules. Overall, this project demonstrates how artificial intelligence can be effectively applied to enhance public safety and decision-making. It serves as a strong step towards building smarter and safer cities, with future scope for integration with real-time data and advanced technologies.

5.2 Future Work

“Future work” refers to the possible improvements and enhancements that can be added to our AI-Smart Public Safety System to make it more efficient, accurate, and scalable. While the current system successfully predicts crime risk, identifies hotspots, and provides features like safe route suggestion and SOS alerts, there is still scope for further development to enhance real-world usability and performance.

One of the major improvements in future is the integration of real-time crime data from official government sources such as police databases or live APIs. This will make the predictions more accurate and dynamic. Additionally, advanced machine learning models like Gradient Boosting or deep learning techniques can be used to improve prediction accuracy and handle more complex patterns. Another important enhancement is the inclusion of real-time GPS tracking and live location sharing during SOS alerts, which can help police respond more quickly and efficiently.

The system can also be extended by developing a mobile application with offline support, making it accessible to a wider range of users. Voice-based alerts and multilingual support can be added to improve usability for people from different regions. Furthermore, integrating external services like emergency helplines, CCTV feeds, and IoT-based surveillance systems can make the platform more powerful and intelligent. These future enhancements will help transform the system into a complete smart city safety solution, improving public security and enabling faster and smarter decision-making.

BIBLIOGRAPHY

- [1] National Crime Records Bureau (NCRB), “Crime in India Reports,” Government of India, 2022. Available: <https://ncrb.gov.in>
- [2] Data.gov.in, “Open Government Data Platform India – Crime Statistics,” Government of India, 2023. Available: <https://data.gov.in>
- [3] Breiman, L., “Random Forests,” *Machine Learning Journal*, vol. 45, no. 1, pp. 5–32, 2001. Available: <https://link.springer.com/article/10.1023/A:1010933404324>
- [4] Ester, M., Kriegel, H.-P., Sander, J., & Xu, X., “A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise (DBSCAN),” *KDD Conference*, 1996. Available: <https://www.aaai.org/Papers/KDD/1996/KDD96-037.pdf>
- [5] Scikit-learn Developers, “Random Forest Classifier Documentation,” 2024. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>
- [6] Flask Developers, “Flask Web Framework Documentation,” 2024. Available: <https://flask.palletsprojects.com>
- [7] React Team, “React – A JavaScript Library for Building User Interfaces,” 2024. Available: <https://react.dev>
- [8] OpenStreetMap Foundation, “OpenStreetMap Data and Services,” 2024. Available: <https://www.openstreetmap.org>
- [9] Chainey, S., & Ratcliffe, J., “GIS and Crime Mapping,” *Wiley Publishing*, 2005.

APPENDIX

App.py(Backend Code)

```
from flask import Flask, jsonify, request
from flask_cors import CORS
import pandas as pd
import numpy as np
import pickle
import json
import sqlite3
import hashlib
import os
from datetime import datetime
app = Flask(__name__)
CORS(app, origins=["http://localhost:5173", "http://localhost:5174"])
# LOAD ML MODEL
model = pickle.load(open("models/risk_model.pkl", "rb"))
le_crime = pickle.load(open("models/le_crime.pkl", "rb"))
le_risk = pickle.load(open("models/le_risk.pkl", "rb"))
with open("models/hotspots.json") as f: hotspots = json.load(f)
with open("models/area_counts.json") as f: area_counts = json.load(f)
df = pd.read_csv("data/crime_data.csv")
AREA_COORDS = {
    "Hitech City": {"lat": 17.4435, "lon": 78.3772},
    "Gachibowli": {"lat": 17.4401, "lon": 78.3489},
    "Banjara Hills": {"lat": 17.4156, "lon": 78.4347},
    "Jubilee Hills": {"lat": 17.4320, "lon": 78.4071},
    "Secunderabad": {"lat": 17.4399, "lon": 78.4983},
    "Charminar": {"lat": 17.3616, "lon": 78.4747},
    "LB Nagar": {"lat": 17.3469, "lon": 78.5469},
    "Dilsukhnagar": {"lat": 17.3688, "lon": 78.5247},
    "Ameerpet": {"lat": 17.4375, "lon": 78.4483},
```

```

    "Kukatpally": {"lat": 17.4849, "lon": 78.4138},
    "Madhapur": {"lat": 17.4485, "lon": 78.3908},
    "Begumpet": {"lat": 17.4418, "lon": 78.4636},
    "Uppal": {"lat": 17.4054, "lon": 78.5590},
    "Miyapur": {"lat": 17.4956, "lon": 78.3694},
    "ECIL": {"lat": 17.4691, "lon": 78.5624},
}

def resolve_coords(lat, lon, area):
    """
    Always use area-centre coordinates when the area name is known.
    This ensures correct police station assignment regardless of
    whether the browser GPS is available or blocked.
    """
    coords = AREA_COORDS.get(area)
    if coords:
        return coords["lat"], coords["lon"]
    return lat, lon

# DATABASE
def get_db():
    conn = sqlite3.connect("data/crime_safety.db")
    conn.row_factory = sqlite3.Row
    return conn

def init_db():
    conn = get_db()
    c = conn.cursor()
    c.execute("""CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        email TEXT UNIQUE NOT NULL,
        password TEXT NOT NULL,
        role TEXT DEFAULT 'public',
        station_id INTEGER DEFAULT 0,
        station_name TEXT DEFAULT NULL
    """)

```

)")

```
c.execute("""CREATE TABLE IF NOT EXISTS police_stations (  
    id      INTEGER PRIMARY KEY AUTOINCREMENT,  
    name    TEXT,  
    area    TEXT,  
    latitude REAL,  
    longitude REAL,  
    phone   TEXT
```

)")

```
c.execute("""CREATE TABLE IF NOT EXISTS sos_alerts (  
    id          INTEGER PRIMARY KEY AUTOINCREMENT,  
    user_name    TEXT,  
    latitude     REAL,  
    longitude    REAL,  
    address     TEXT,  
    area        TEXT,  
    assigned_station TEXT,  
    assigned_station_id INTEGER,  
    status      TEXT DEFAULT 'active',  
    timestamp   DATETIME DEFAULT CURRENT_TIMESTAMP
```

)")

```
c.execute("""CREATE TABLE IF NOT EXISTS incidents (  
    id          INTEGER PRIMARY KEY AUTOINCREMENT,  
    user_name    TEXT,  
    crime_type   TEXT,  
    description  TEXT,  
    latitude     REAL,  
    longitude    REAL,  
    area        TEXT,  
    severity     TEXT,  
    assigned_station TEXT,  
    assigned_station_id INTEGER,  
    status      TEXT DEFAULT 'pending',
```

```

        timestamp    DATETIME DEFAULT CURRENT_TIMESTAMP
    )")

c.execute("""CREATE TABLE IF NOT EXISTS alerts (
    id    INTEGER PRIMARY KEY AUTOINCREMENT,
    area  TEXT,
    message TEXT,
    severity TEXT,
    station_id INTEGER DEFAULT 0,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
)""")

c.execute("""CREATE TABLE IF NOT EXISTS route_history (
    id    INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER,
    source TEXT,
    destination TEXT,
    risk_score TEXT,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
)""")

# Seed police stations — all 15 areas
c.execute("SELECT COUNT(*) FROM police_stations")
if c.fetchone()[0] == 0:
    stations = [
        ("Hitech City PS", "Hitech City", 17.4460, 78.3780, "040-23304986"),
        ("Gachibowli PS", "Gachibowli", 17.4410, 78.3500, "040-23304987"),
        ("Banjara Hills PS", "Banjara Hills", 17.4160, 78.4350, "040-23304988"),
        ("Jubilee Hills PS", "Jubilee Hills", 17.4325, 78.4075, "040-23305000"),
        ("Secunderabad PS", "Secunderabad", 17.4400, 78.4990, "040-23304989"),
        ("Charminar PS", "Charminar", 17.3620, 78.4750, "040-23304990"),
        ("LB Nagar PS", "LB Nagar", 17.3470, 78.5470, "040-23304991"),
        ("Dilsukhnagar PS", "Dilsukhnagar", 17.3690, 78.5250, "040-23304996"),
        ("Ameerpet PS", "Ameerpet", 17.4380, 78.4490, "040-23304993"),
        ("Kukatpally PS", "Kukatpally", 17.4850, 78.4140, "040-23304992"),
        ("Madhapur PS", "Madhapur", 17.4490, 78.3910, "040-23304995"),
    ]

```

```

        ("Begumpet PS",    "Begumpet",    17.4420, 78.4640, "040-23304997"),
        ("Uppal PS",      "Uppal",        17.4060, 78.5590, "040-23304994"),
        ("Miyapur PS",    "Miyapur",    17.4960, 78.3700, "040-23304998"),
        ("ECIL PS",       "ECIL",         17.4695, 78.5625, "040-23304999"),
    ]
    c.executemany(
        "INSERT INTO police_stations (name,area,latitude,longitude,phone) VALUES
        (?, ?, ?, ?, ?)",
        stations
    )
    # Seed alerts
    c.execute("SELECT COUNT(*) FROM alerts")
    if c.fetchone()[0] == 0:
        seed_alerts = [
            ("Charminar", " ⚠️ High theft activity near Charminar market", "High", 5),
            ("LB Nagar", " 🚒 Chain snatching at LB Nagar bus stop", "High", 6),
            ("Secunderabad", " ⚠️ Vehicle theft near railway station", "Medium", 4),
            ("Uppal", " 💎 Robbery reported near Uppal crossroads", "High", 13),
            ("Ameerpet", " 🚔 Pickpocket incidents near metro station", "Medium", 9),
        ]
        c.executemany(
            "INSERT INTO alerts (area,message,severity,station_id) VALUES (?, ?, ?, ?)",
            seed_alerts
        )
    conn.commit()
    conn.close()
init_db()
def hash_password(p):
    return hashlib.sha256(p.encode()).hexdigest()
# AUTH
@app.route("/api/register", methods=["POST"])
def register():

```

```

data      = request.json
name      = data.get("name")
email     = data.get("email")
password  = hash_password(data.get("password"))
role      = data.get("role", "public")
station_id = data.get("station_id", 0)
station_name = data.get("station_name", None)
if role == "police" and not station_id:
    return jsonify({"success": False,
                    "message": "Police must select a station!"}), 400
try:
    conn = get_db()
    conn.execute(
        """INSERT INTO users
           (name,email,password,role,station_id,station_name)
           VALUES (?, ?, ?, ?, ?, ?)""",
        (name, email, password, role, station_id, station_name)
    )
    conn.commit()
    conn.close()
    return jsonify({"success": True, "message": "Registered successfully"})
except sqlite3.IntegrityError:
    return jsonify({"success": False,
                    "message": "Email already exists"}), 400
@app.route("/api/login", methods=["POST"])
def login():
    data = request.json
    conn = get_db()
    user = conn.execute(
        "SELECT * FROM users WHERE email=? AND password=?",
        (data["email"], hash_password(data["password"]))
    ).fetchone()
    conn.close()

```

```

if user:
    return jsonify({
        "success": True,
        "user": {
            "id": user["id"],
            "name": user["name"],
            "role": user["role"],
            "station_id": user["station_id"],
            "station_name": user["station_name"]
        }
    })
return jsonify({"success": False,
                "message": "Invalid credentials"}), 401

# STATIONS LIST
@app.route("/api/police-stations-list", methods=["GET"])
def police_stations_list():
    conn = get_db()
    stations = conn.execute(
        "SELECT id, name, area FROM police_stations"
    ).fetchall()
    conn.close()
    return jsonify({"success": True,
                    "stations": [dict(s) for s in stations]})

# PUBLIC APIs
@app.route("/api/hotspots", methods=["GET"])
def get_hotspots():
    return jsonify({"success": True, "hotspots": hotspots})

@app.route("/api/areas", methods=["GET"])
def get_areas():
    return jsonify({"success": True,
                    "areas": df["area"].unique().tolist()})

@app.route("/api/area-risk", methods=["GET"])
def area_risk():

```

```

area    = request.args.get("area", "")
hour    = int(request.args.get("hour", 12))
count   = area_counts.get(area, 10)
enc     = le_crime.transform(["Theft"])[0]
is_night = 1 if (hour >= 20 or hour <= 5) else 0
coords  = AREA_COORDS.get(area, {"lat": 17.44, "lon": 78.44})
features = np.array([[coords["lat"], coords["lon"], hour, 6, 2,
                        is_night, enc, 1, count]])
pred    = model.predict(features)[0]
risk    = le_risk.inverse_transform([pred])[0]
area_df = df[df["area"] == area]
top_crime = area_df["crime_type"].mode()[0] \
            if len(area_df) > 0 else "Unknown"
return jsonify({
    "success": True,
    "area": area,
    "risk_level": risk.capitalize(),
    "crime_count": count,
    "top_crime": top_crime,
    "hour": hour
}))
@app.route("/api/nearby-police", methods=["GET"])
def nearby_police():
    lat    = float(request.args.get("lat", 17.44))
    lon    = float(request.args.get("lon", 78.44))
    area    = request.args.get("area", "")
    lat, lon = resolve_coords(lat, lon, area)
    conn    = get_db()
    stations = conn.execute(
        "SELECT * FROM police_stations"
    ).fetchall()
    conn.close()
    results = []

```

```

for s in stations:
    dist = ((s["latitude"] - lat)**2 +
            (s["longitude"] - lon)**2)**0.5
    results.append({
        "name":    s["name"],
        "area":    s["area"],
        "latitude": s["latitude"],
        "longitude": s["longitude"],
        "phone":    s["phone"],
        "distance": round(dist * 111, 2)
    })
results = sorted(results, key=lambda x: x["distance"])[0:5]
return jsonify({"success": True, "stations": results})

@app.route("/api/crime-stats", methods=["GET"])
def crime_stats():
    crime_counts = df["crime_type"].value_counts().to_dict()
    area_counts_s = df["area"].value_counts().head(10).to_dict()
    df["hour_col"] = df["time"].str.split(":").str[0].astype(int)
    hour_counts = df["hour_col"].value_counts().sort_index().to_dict()
    return jsonify({
        "success":    True,
        "crime_types": crime_counts,
        "top_areas":  area_counts_s,
        "hourly":     {str(k): v for k, v in hour_counts.items()}
    })

# SAFE ROUTE
@app.route("/api/safe-route", methods=["POST"])
def safe_route():
    data = request.json
    source = data.get("source", "").strip()
    dest = data.get("destination", "").strip()
    hour = datetime.now().hour
    is_night = hour >= 20 or hour <= 5

```

```

def match_area(user_input):
    user_input = user_input.strip().lower()
    for area in AREA_COORDS:
        if area.lower() == user_input:
            return area
    for area in AREA_COORDS:
        if user_input in area.lower() or area.lower() in user_input:
            return area
    return None

source_area = match_area(source)
dest_area = match_area(dest)
if not source_area or not dest_area:
    return jsonify({
        "success": False,
        "message": f"Area not found. Available: {' '.join(AREA_COORDS.keys())}"
    }), 400

source_coord = AREA_COORDS[source_area]
dest_coord = AREA_COORDS[dest_area]
df["hour_col"] = df["time"].str.split(":").str[0].astype(int)
def get_real_crimes_near(coord, radius_km=3.0):
    lat = coord["lat"]
    lon = coord["lon"]
    df["dist"] = ((df["latitude"] - lat)**2 +
                  (df["longitude"] - lon)**2)**0.5 * 111
    return df[df["dist"] <= radius_km].copy()
source_crimes_df = get_real_crimes_near(source_coord)
dest_crimes_df = get_real_crimes_near(dest_coord)
def time_weighted_score(crimes_df):
    if len(crimes_df) == 0:
        return 0
    same_hour = crimes_df[
        (crimes_df["hour_col"] >= max(0, hour - 2)) &
        (crimes_df["hour_col"] <= min(23, hour + 2))

```

```

    ]
    return len(crimes_df) + (len(same_hour) * 2)
source_score = time_weighted_score(source_crimes_df)
dest_score = time_weighted_score(dest_crimes_df)
def predict_risk_ml(coord, crime_count):
    enc = le_crime.transform(["Theft"])[0]
    is_n = 1 if is_night else 0
    features = np.array([
        coord["lat"], coord["lon"],
        hour, datetime.now().month,
        datetime.now().weekday(),
        is_n, enc, 1, crime_count
    ])
    pred = model.predict(features)[0]
    return le_risk.inverse_transform([pred])[0]
source_ml_risk = predict_risk_ml(source_coord, len(source_crimes_df))
dest_ml_risk = predict_risk_ml(dest_coord, len(dest_crimes_df))
risk_map = {"high": 3, "medium": 2, "low": 1}
ml_score = (risk_map.get(source_ml_risk, 1) +
            risk_map.get(dest_ml_risk, 1)) * 15
raw_score = min(source_score + dest_score + ml_score, 200)
risk_score = min(int(raw_score / 2), 100)
if is_night:
    risk_score = min(int(risk_score * 1.4), 100)
if risk_score >= 60:
    risk = "High"
elif risk_score >= 30:
    risk = "Medium"
else:
    risk = "Low"
def crime_breakdown(crimes_df):
    if len(crimes_df) == 0:
        return {}

```

```

    return crimes_df["crime_type"].value_counts().head(5).to_dict()
def peak_hours(crimes_df):
    if len(crimes_df) == 0:
        return []
    top = crimes_df["hour_col"].value_counts().head(3).index.tolist()
    return [f'{h}:00' for h in sorted(top)]
source_breakdown = crime_breakdown(source_crimes_df)
dest_breakdown = crime_breakdown(dest_crimes_df)
source_peak = peak_hours(source_crimes_df)
dest_peak = peak_hours(dest_crimes_df)
# — FIXED: radius 2km + sort by distance not crime_count —
def nearby_hotspots_fn(coord, radius_km=2.0):
    results = []
    for h in hotspots:
        dist = ((h["latitude"] - coord["lat"])**2 +
                (h["longitude"] - coord["lon"])**2)**0.5 * 111
        if dist <= radius_km:
            results.append({**h, "distance_km": round(dist, 2)})
    return sorted(results, key=lambda x: x["distance_km"])
route_hotspots = (nearby_hotspots_fn(source_coord) +
                  nearby_hotspots_fn(dest_coord))
seen = set()
unique_hotspots = []
for h in route_hotspots:
    if h["cluster_id"] not in seen:
        seen.add(h["cluster_id"])
        unique_hotspots.append(h)
# Sort by distance so closest hotspots show first
unique_hotspots = sorted(unique_hotspots,
                          key=lambda x: x["distance_km"])[0:6]
all_crimes = (list(source_breakdown.keys()) +
              list(dest_breakdown.keys()))
tips = []

```

```

if "Chain Snatching" in all_crimes or "Robbery" in all_crimes:
    tips.append(" ⚠️ Chain snatching reported — avoid wearing jewellery")
if "Vehicle Theft" in all_crimes:
    tips.append(" 🚗 Vehicle theft common — park only in guarded areas")
if "Harassment" in all_crimes:
    tips.append(" 👥 Travel in groups, especially after dark")
if "Pickpocket" in all_crimes:
    tips.append(" 👜 Keep bags in front, avoid crowded spots")
if is_night:
    tips.append(" 🌙 Late night — share live location with someone")
tips.append(" 📞 Emergency: Police 100 | Women Helpline 1091")
if not tips:
    tips.append(" ✅ Stay aware and keep valuables secure")
conn = get_db()
stations = conn.execute(
    "SELECT * FROM police_stations"
).fetchall()
conn.close()
nearest_ps = min(
    [dict(s) for s in stations],
    key=lambda s: ((s["latitude"] - dest_coord["lat"])**2 +
                    (s["longitude"] - dest_coord["lon"])**2)**0.5
)
return jsonify({
    "success": True,
    "source": source_area,
    "destination": dest_area,
    "risk_level": risk,
    "risk_score": risk_score,
    "source_ml_risk": source_ml_risk.capitalize(),
    "dest_ml_risk": dest_ml_risk.capitalize(),
    "advice": (

```

```

f" 🚨 HIGH RISK — {len(source_crimes_df)} crimes near "
f"{source_area}, {len(dest_crimes_df)} crimes near {dest_area}."
if risk == "High" else

f" 🟡 MODERATE — Some crime activity near this route."
if risk == "Medium" else

f" 🟢 SAFE — Low crime activity near both areas."
),
"safety_tips": tips,
"source_crime_count": int(len(source_crimes_df)),
"dest_crime_count": int(len(dest_crimes_df)),
"source_breakdown": source_breakdown,
"dest_breakdown": dest_breakdown,
"source_peak_hours": source_peak,
"dest_peak_hours": dest_peak,
"hotspots_nearby": unique_hotspots,
"nearest_police": nearest_ps,
"checked_at": f"{hour:02d}:00 hrs",
"is_night": is_night,
"night_warning": (
    f" 🌙 It is {hour:02d}:00 hrs — crime rates are higher at night!"
    if is_night else ""
)
})
# SOS
@app.route("/api/sos", methods=["POST"])
def sos():
    data = request.json
    raw_lat = float(data.get("latitude", 17.44))
    raw_lon = float(data.get("longitude", 78.44))
    area = data.get("area", "Unknown")
    user_name = data.get("user_name", "Unknown")
    address = data.get("address", "Unknown location")

```

```

user_lat, user_lon = resolve_coords(raw_lat, raw_lon, area)
conn = get_db()
stations = conn.execute(
    "SELECT * FROM police_stations"
).fetchall()
nearest = min(
    [dict(s) for s in stations],
    key=lambda s: ((s["latitude"] - user_lat)**2 +
                    (s["longitude"] - user_lon)**2)**0.5
)
conn.execute(
    """INSERT INTO sos_alerts
    (user_name,latitude,longitude,address,area,
    assigned_station,assigned_station_id,status)
    VALUES (?,?,?,?,?,?,?, 'active')""",
    (user_name, user_lat, user_lon, address, area,
    nearest["name"], nearest["id"])
)
conn.commit()
conn.close()
return jsonify({
    "success": True,
    "message": f"🚒 SOS sent! {nearest['name']} has been notified.",
    "helpline": "100",
    "nearest_station": nearest["name"],
    "station_phone": nearest["phone"],
    "station_area": nearest["area"]
})
@app.route("/api/sos-alerts", methods=["GET"])
def get_sos_alerts():
    station_id = request.args.get("station_id")
    conn = get_db()
    if station_id and station_id != "0":

```

```

        alerts = conn.execute(
            """SELECT * FROM sos_alerts
            WHERE assigned_station_id=?
            ORDER BY timestamp DESC LIMIT 20""",
            (station_id,)
        ).fetchall()
    else:
        alerts = conn.execute(
            "SELECT * FROM sos_alerts ORDER BY timestamp DESC LIMIT 20"
        ).fetchall()
    conn.close()
    return jsonify({"success": True,
                    "alerts": [dict(a) for a in alerts]})

@app.route("/api/sos-resolve/<int:alert_id>", methods=["POST"])
def resolve_sos(alert_id):
    conn = get_db()
    conn.execute(
        "UPDATE sos_alerts SET status='resolved' WHERE id=?",
        (alert_id,)
    )
    conn.commit()
    conn.close()
    return jsonify({"success": True, "message": "Alert resolved"})

# LIVE ALERTS
@app.route("/api/alerts", methods=["GET"])
def get_alerts():
    station_id = request.args.get("station_id")
    conn = get_db()
    if station_id and station_id != "0":
        alerts = conn.execute(
            """SELECT * FROM alerts
            WHERE station_id=? OR station_id=0
            ORDER BY timestamp DESC LIMIT 10""",

```

```

        (station_id,)
    ).fetchall()
else:
    alerts = conn.execute(
        "SELECT * FROM alerts ORDER BY timestamp DESC LIMIT 10"
    ).fetchall()
conn.close()
return jsonify({"success": True,
                "alerts": [dict(a) for a in alerts]})

# INCIDENT REPORTING
@app.route("/api/report-incident", methods=["POST"])
def report_incident():
    data = request.json
    raw_lat = float(data.get("latitude", 17.44))
    raw_lon = float(data.get("longitude", 78.44))
    area = data.get("area", "Unknown")
    user_name = data.get("user_name", "Anonymous")
    user_lat, user_lon = resolve_coords(raw_lat, raw_lon, area)
    conn = get_db()
    stations = conn.execute(
        "SELECT * FROM police_stations"
    ).fetchall()
    nearest = min(
        [dict(s) for s in stations],
        key=lambda s: ((s["latitude"] - user_lat)**2 +
                        (s["longitude"] - user_lon)**2)**0.5
    )
    conn.execute(
        """INSERT INTO incidents
        (user_name,crime_type,description,latitude,longitude,
        area,severity,assigned_station,assigned_station_id,status)
        VALUES (?,?,?,?,?,?,?,?,'pending')""",
        (user_name, data.get("crime_type", "Other"),

```

```

        data.get("description", ""),
        user_lat, user_lon, area,
        data.get("severity", "Medium"),
        nearest["name"], nearest["id"])
    )
    conn.execute(
        """INSERT INTO alerts (area,message,severity,station_id)
        VALUES (?, ?, ?, ?)""",
        (area,
        f"🚒 Incident: {data.get('crime_type','Crime')} near {area} by {user_name}",
        data.get("severity", "Medium"),
        nearest["id"])
    )
    conn.commit()
    conn.close()
    return jsonify({
        "success": True,
        "message": f"✅ Report sent to {nearest['name']}!",
        "assigned_to": nearest["name"],
        "station_phone": nearest["phone"]
    })
@app.route("/api/incidents", methods=["GET"])
def get_incidents():
    station_id = request.args.get("station_id")
    conn = get_db()
    if station_id and station_id != "0":
        incidents = conn.execute(
            """SELECT * FROM incidents
            WHERE assigned_station_id=?
            ORDER BY timestamp DESC LIMIT 30""",
            (station_id,)
        ).fetchall()
    else:

```

```

incidents = conn.execute(
    "SELECT * FROM incidents ORDER BY timestamp DESC LIMIT 30"
).fetchall()
conn.close()
return jsonify({"success": True,
                "incidents": [dict(i) for i in incidents]})

# PATROL RECOMMENDATIONS
@app.route("/api/patrol-recommendations", methods=["GET"])
def patrol_recommendations():
    hour = datetime.now().hour
    is_night = hour >= 20 or hour <= 5
    is_peak = 17 <= hour <= 20
    recommendations = []
    for h in hotspots[:10]:
        area_df = df[df["area"] == h["area"]]
        area_df = area_df.copy()
        area_df["h"] = area_df["time"].str.split(":").str[0].astype(int)
        night_crimes = len(area_df[area_df["h"] >= 20])
        day_crimes = len(area_df) - night_crimes
        top_crimes = area_df["crime_type"].value_counts().head(3).to_dict()
        unresolved = len(area_df[area_df["status"] == "Unresolved"]) \
            if "status" in area_df.columns else 0
        if h["risk_level"] == "High":
            priority = "🔴 URGENT"
            action = "Deploy 2 patrol units immediately"
            frequency = "Every 30 minutes"
        elif h["risk_level"] == "Medium":
            priority = "🟡 MODERATE"
            action = "Increase patrol frequency"
            frequency = "Every 1 hour"
        else:
            priority = "🟢 NORMAL"

```

```

        action = "Regular patrol"
        frequency = "Every 2 hours"
        time_warning = ""
        if is_night and night_crimes > day_crimes:
            time_warning = " ⚠️ Night hotspot — double patrol tonight"
        elif is_peak:
            time_warning = " ⚠️ Peak hours — monitor closely"
        recommendations.append({
            "area": h["area"],
            "risk_level": h["risk_level"],
            "priority": priority,
            "action": action,
            "frequency": frequency,
            "crime_count": h["crime_count"],
            "top_crimes": list(top_crimes.keys()),
            "unresolved": unresolved,
            "time_warning": time_warning,
            "night_crimes": night_crimes,
            "day_crimes": day_crimes,
            "coordinates": {"lat": h["latitude"], "lon": h["longitude"]}
        })
    return jsonify({
        "success": True,
        "current_hour": hour,
        "shift": ("Night Shift 🌙" if is_night
                 else "Evening Shift 🌃" if is_peak
                 else "Day Shift ☀️"),
        "recommendations": recommendations
    })

# TONIGHT'S RISK PREDICTION
@app.route("/api/risk-prediction-tonight", methods=["GET"])
def risk_prediction_tonight():

```

```

hour      = datetime.now().hour
predictions = []
for area_name, count in list(area_counts.items())[:10]:
    coords = AREA_COORDS.get(area_name, {"lat": 17.44, "lon": 78.44})
    enc     = le_crime.transform(["Theft"])[0]
    features = np.array([[coords["lat"], coords["lon"], 21, 6, 4, 1, enc, 1, count]])
    pred     = model.predict(features)[0]
    risk     = le_risk.inverse_transform([pred])[0]
    area_df = df[df["area"] == area_name].copy()
    area_df["h"] = area_df["time"].str.split(":").str[0].astype(int)
    night_count = len(area_df[area_df["h"] >= 20])
    predictions.append({
        "area":          area_name,
        "predicted_risk": risk.capitalize(),
        "night_crime_history": night_count,
        "total_crimes":   count,
        "alert":          night_count > 20
    })
predictions.sort(key=lambda x: x["night_crime_history"], reverse=True)
return jsonify({
    "success":    True,
    "predictions": predictions,
    "generated_at": datetime.now().strftime("%H:%M:%S")
})
# RUN
if __name__ == "__main__":
    app.run(debug=True, port=5000)

```

ML_model.py

```

import pandas as pd
import numpy as np
from sklearn.cluster import DBSCAN
from sklearn.ensemble import RandomForestClassifier

```

```

from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score
from sklearn.preprocessing import LabelEncoder
import pickle
import os
import json

# 1. LOAD DATA
print("📁 Loading dataset...")
df = pd.read_csv("data/crime_data.csv")
print(f"✅ Loaded {len(df)} records\n")

# 2. FEATURE ENGINEERING
# Extract hour and month from date/time
df["hour"] = df["time"].str.split(":").str[0].astype(int)
df["month"] = pd.to_datetime(df["date"]).dt.month
df["weekday"] = pd.to_datetime(df["date"]).dt.dayofweek
df["is_night"] = (df["hour"] >= 20) | (df["hour"] <= 5)
df["is_night"] = df["is_night"].astype(int)
# Encode crime type as number
le_crime = LabelEncoder()
df["crime_type_encoded"] = le_crime.fit_transform(df["crime_type"])
# Encode risk label
le_risk = LabelEncoder()
df["risk_encoded"] = le_risk.fit_transform(df["base_risk"])
# Count crimes per area (density feature)
area_counts = df["area"].value_counts().to_dict()
df["area_crime_count"] = df["area"].map(area_counts)
print("Features ready\n")

# 3. DBSCAN HOTSPOT DETECTION
print("Running DBSCAN hotspot detection...")
# Convert lat/lon to radians for DBSCAN
coords = df[["latitude", "longitude"]].values
coords_rad = np.radians(coords)
# DBSCAN: eps=0.5km, min 5 crimes to form a hotspot

```

```

db = DBSCAN(eps=1.5/6371, min_samples=3, algorithm="ball_tree", metric="haversine")
df["cluster"] = db.fit_predict(coords_rad)
# Count how many clusters found
n_clusters = len(set(df["cluster"])) - (1 if -1 in df["cluster"].values else 0)
print(f" Found {n_clusters} crime hotspot clusters\n")
# Build hotspot summary per cluster
hotspots = []
for cluster_id in sorted(set(df["cluster"])):
    if cluster_id == -1:
        continue # skip noise points
    cluster_data = df[df["cluster"] == cluster_id]
    hotspots.append({
        "cluster_id": int(cluster_id),
        "latitude": round(float(cluster_data["latitude"].mean()), 6),
        "longitude": round(float(cluster_data["longitude"].mean()), 6),
        "crime_count": int(len(cluster_data)),
        "top_crime": cluster_data["crime_type"].mode()[0],
        "area": cluster_data["area"].mode()[0],
        "risk_level": "High" if len(cluster_data) > 20
                      else "Medium" if len(cluster_data) > 10
                      else "Low"
    })
# Sort by crime count
hotspots = sorted(hotspots, key=lambda x: x["crime_count"], reverse=True)
# Save hotspots to JSON
os.makedirs("models", exist_ok=True)
with open("models/hotspots.json", "w") as f:
    json.dump(hotspots, f, indent=2)
print(f" Saved {len(hotspots)} hotspots to models/hotspots.json\n")
# 4. RANDOM FOREST RISK PREDICTION
print("🤖 Training Random Forest model...")
features = [
    "latitude", "longitude", "hour", "month",

```

```

    "weekday", "is_night", "crime_type_encoded",
    "severity_score", "area_crime_count"
]
X = df[features]
y = df["risk_encoded"] # 0=high, 1=low, 2=medium (alphabetical)
# Split into train and test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Train the model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
# Evaluate
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f" Model Accuracy: {accuracy * 100:.2f}%\n")
print(" Classification Report:")
print(classification_report(y_test, y_pred,
    target_names=le_risk.classes_))
# 5. SAVE MODEL & ENCODERS
print("\n Saving model and encoders...")
pickle.dump(model, open("models/risk_model.pkl", "wb"))
pickle.dump(le_crime, open("models/le_crime.pkl", "wb"))
pickle.dump(le_risk, open("models/le_risk.pkl", "wb"))
# Save area crime counts for API use
with open("models/area_counts.json", "w") as f:
    json.dump(area_counts, f)
print(" Saved: models/risk_model.pkl")
print("Saved: models/le_crime.pkl")
print(" Saved: models/le_risk.pkl")
print("Saved: models/area_counts.json")
print("\nML Phase Complete! All models ready.")

```